

Lab 7 Build Guide

Lab 7 Build Guide: AI-Assisted Network Operations

This guide walks through connecting an LLM to the Network as Code pipeline you built in Labs 1 through 6. By the end you will have an MCP server that exposes every NaC tool to AI clients, an assistant that answers network operations questions using live fabric data, and the option to run everything with a local open-source model instead of a cloud API.

What We Built

Labs 1 through 6 created a complete automation stack: data model, validation, config generation, deployment, testing, and drift detection. Every one of those capabilities is a Python module with structured input and output. Lab 7 connects an LLM to those modules so you can interact with the fabric through natural language.

There are two pieces. The MCP server wraps every tool we built as an MCP tool, making them callable by Claude Code or any MCP-compatible client. The AI assistant is a standalone script that gathers fabric data, sends it to an LLM with a question, and returns a useful answer.

The assistant supports two LLM backends. Claude via the Anthropic API for people who have API access, and Ollama running Llama 3.2 locally for people who want a free, private, no-API-key option. Both backends use the same prompts and produce comparable results. The choice is a command-line flag.

Prerequisites

You need Labs 1-6 complete with the fabric deployed, all tests passing, and no drift.

```
uv run python -m scripts.fabric_status
uv run python -m drift.detect
```

All 6 devices UP, no drift detected.

Part 1: Install Ollama and Pull a Model

Ollama is an open-source tool that runs LLMs locally. It supports dozens of models including Llama, Mistral, and Gemma. We use Llama 3.2 3B because it fits in 4GB of RAM and runs on CPU.

```
curl -fsSL https://ollama.com/install.sh | sudo sh
```

Verify it is running:

```
ollama --version
```

Pull the Llama 3.2 3B model:

```
ollama pull llama3.2:3b
```

This downloads about 2GB. After it finishes, verify the model is available:

```
ollama list
```

You should see llama3.2:3b in the list. Test it with a quick prompt:

```
ollama run llama3.2:3b "What is BGP in one sentence?"
```

If you get a response about the Border Gateway Protocol, Ollama is working.

Part 2: The MCP Server

The MCP server lives at `agent/mcp_server.py`. It exposes 8 tools that map directly to the modules we built in earlier labs.

```
cat agent/mcp_server.py
```

What MCP Is

The Model Context Protocol (MCP) is a standard for connecting AI models to external tools. Instead of hardcoding tool calls into your application, you define tools with names, descriptions, and input schemas. Any MCP client can discover and call those tools. It is the interface layer between the AI and your infrastructure.

The 8 Tools

Tool	Maps to	What it does
fabric_status	scripts.fabric_status	OSPF/BGP/interface health for all devices
validate	validate.py	Run 39 validation checks on the data model
detect_drift	drift.detect	Compare intended vs running configs
generate_configs	generators.python.render	

Tool	Maps to	What it does
		Render FRR configs from the data model
deploy_configs	deploy.scrapli_deploy	Push configs to ContainerLab devices
ping_mesh	scripts.ping_mesh	Loopback-to-loopback reachability test
route_table	scripts.route_table	Routing table viewer with protocol filtering
run_tests	pytest tests/post_change/	31 post-change validation tests

Each tool is a thin wrapper that calls the existing module and returns the output as text. The MCP server does not contain any logic itself. All the logic lives in the modules you already built and tested.

Using with Claude Code

To use the MCP server with Claude Code, add it to your MCP configuration. In your Claude Code settings, add:

```
{
  "mcpServers": {
    "nac-fabric": {
      "command": "uv",
      "args": ["run", "python", "-m", "agent.mcp_server"],
      "cwd": "/home/elliott/network-as-code-labs"
    }
  }
}
```

Once configured, Claude Code can call any of the 8 tools by name. You can ask it to check the fabric health, run validation, detect drift, or deploy configs, and it will use the MCP tools to do it.

Testing the Server

Verify the server starts and lists all tools:

```
uv run python -c "
import asyncio
from agent.mcp_server import list_tools

async def test():
    tools = await list_tools()
    for t in tools:
        print(f'  {t.name}: {t.description[:60]}...')
    print(f'\n{len(tools)} tools available')

asyncio.run(test())
"
```

You should see all 8 tools listed.

Part 3: The AI Assistant

The assistant lives at `agent/assistant.py`. It has three modes and two backends.

```
cat agent/assistant.py
```

Backend Selection

The `--backend` flag switches between Claude and Ollama:

```
# Use Claude API (requires ANTHROPIC_API_KEY env var)
uv run python -m agent.assistant --backend claude fabric-qa "Is
the fabric healthy?"

# Use local Llama (free, no API key)
uv run python -m agent.assistant --backend ollama fabric-qa "Is
the fabric healthy?"
```

If you have a Claude API key, set it:

```
export ANTHROPIC_API_KEY='your-key-here'
```

If you do not have one, use `--backend ollama` for everything. The results are comparable for network operations questions.

Mode 1: Validation Assistant

When a validation check fails, the assistant explains what went wrong and suggests a fix.

To demo this, first introduce a validation error. Edit `data/overlay.yaml` and change the keepalive to 1 (which will fail the BGP holdtime $\geq 3 \times$ keepalive compliance check since $90 < 3$):

Actually, change the holdtime to 2 which is less than 3x keepalive (30):

```
bgp:
  address_families:
    - evpn
    - ipv4_unicast
  timers:
    keepalive: 30
    holdtime: 2
```

Now run the validation assistant:

```
uv run python -m agent.assistant --backend ollama validation-
assist
```

The assistant runs validation, sees the failure, sends it to Llama, and gets back an explanation: the holdtime must be at least 3x the keepalive interval (RFC 4271), and you need to set it back to 90 or higher.

Revert the change after the demo.

Mode 2: Fabric Q&A

Ask natural language questions about the fabric. The assistant gathers live data from the devices and sends it to the LLM for interpretation.

```
uv run python -m agent.assistant --backend ollama fabric-qa  
"Which device has the most BGP peers?"
```

The answer should identify spine1 and spine2 with 5 peers each (every other device), while the leafs and borders have 2 peers each (the two spines).

More examples:

```
uv run python -m agent.assistant --backend ollama fabric-qa "Are  
all OSPF adjacencies healthy?"  
uv run python -m agent.assistant --backend ollama fabric-qa "What  
is the total route count across the fabric?"  
uv run python -m agent.assistant --backend ollama fabric-qa  
"Explain the routing path from leaf1 to border2"
```

Mode 3: Drift Triage

Introduce drift on a device, then ask the LLM to classify it and recommend a reconciliation strategy.

```
docker exec clab-nac-spine-leaf-spine1 vtysh -c "conf t" -c  
"router bgp 65000" -c "no bgp cluster-id" -c "end"
```

Now run drift triage:

```
uv run python -m agent.assistant --backend ollama drift-triage
```

The assistant detects the drift, sends the report to Llama, and gets back an analysis: the missing cluster-id looks like either a mistake or an emergency fix, the risk of auto-remediation is medium, and the recommended strategy is to report for review before remediating.

Restore the config after the demo:

```
uv run python -m drift.reconcile remediate --device spine1
```

Part 4: Claude vs Ollama – What to Expect

Both backends work for all three modes. The differences:

Claude (via API) is faster, more articulate, and better at nuanced analysis. It costs money per API call but the cost is minimal for network operations queries.

Ollama/Llama is free, runs locally, and keeps all data on your machine. It is slower (especially on CPU) and less polished in its answers, but it handles structured network data well enough for practical use.

For a YouTube demo, Ollama is the better choice because your viewers can follow along without an API key. For production use, Claude gives better results.

You can also swap models in Ollama. If you have more RAM or a GPU:

```
ollama pull llama3.1:8b
uv run python -m agent.assistant --backend ollama --model
    llama3.1:8b fabric-qa "Is the fabric healthy?"
```

The `--model` flag lets you try different models without changing any code.

Part 5: Commit Lab 7

```
git status
```

You should see the new `agent/` directory with 3 files, updated `pyproject.toml` and `uv.lock` (with `anthropic`, `httpx`, `mcp` SDKs), and the Lab 7 docs.

```
git add agent/ pyproject.toml uv.lock docs/lab7-build-guide.md
git commit -m "Lab 7: MCP server, AI assistant with Claude and
    Ollama backends"
```

Check the log:

```
git log --oneline
```

Seven commits. Seven labs. The automation stack is complete from YAML intent to AI-assisted operations.

Part 6: What We Proved

By the end of this lab you have demonstrated three things.

First, that MCP is the interface layer between AI and infrastructure. The 8 tools we exposed are the same modules we built and tested in Labs 1 through 6. The MCP server adds no new logic. It simply makes existing capabilities discoverable and callable by AI clients. Any MCP-compatible tool (Claude Code, custom agents, future products) can interact with the fabric through this server.

Second, that you are not locked into a single LLM vendor. The same prompts, the same tools, the same workflow works with Claude and with a local Llama model. The backend is a flag. This matters because the network operations logic is in your Python code, not in the LLM. The LLM adds natural language interpretation and analysis on top of structured data that your tools already collect.

Third, that AI assistance is practical today for network operations. The validation assistant explains failures in plain English. The fabric Q&A answers questions without requiring you to memorize CLI commands. The drift triage adds judgment to automated detection. None of these replace the engineer. They augment the engineer by translating between structured data and human understanding.

Troubleshooting

Ollama “connection refused”: Make sure Ollama is running. Check with `systemctl status ollama` or start it with `ollama serve` in another terminal.

Ollama is slow: The 3B model runs on CPU in about 10-30 seconds per response. If you need faster responses, use a GPU or switch to the Claude backend.

Claude API “key not set”: Set the environment variable with `export ANTHROPIC_API_KEY='your-key'`. The key is never stored in code.

MCP server won’t start: Check that all dependencies are installed with `uv sync`. The MCP SDK requires Python 3.11+.

Assistant returns “(empty response)”: The LLM may have hit a timeout or returned an unexpected format. Try again or switch backends. Ollama occasionally returns empty on the first request after starting.

Model pull fails: Check internet connectivity. `ollama pull` downloads from the Ollama registry. If you are behind a proxy, configure it in your environment.